

Abstract Data Type

ADT Insieme

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio



Insieme (Set)

Insieme (Set)

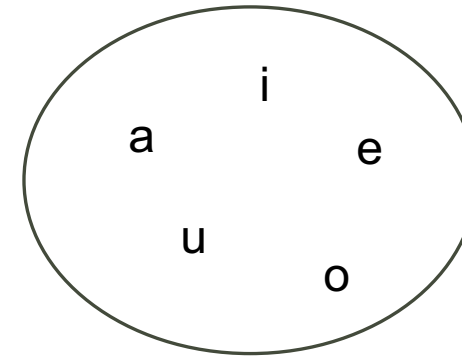


- Un insieme (set) è:
 - una collezione non ordinata di elementi senza duplicati
 -
 -
 -
 -
 -

Insieme (Set)



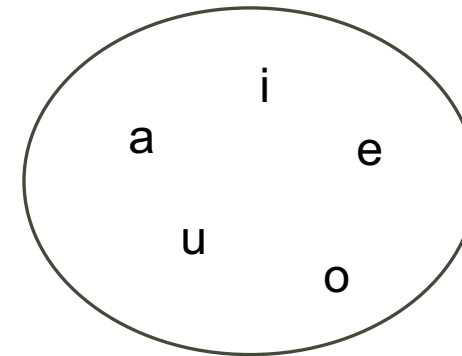
- Un insieme (set) è:
 - una collezione non ordinata di elementi senza duplicati
 - Esempio:
 - Le vocali: {'a', 'e', 'i', 'o', 'u'}
-
-
-



Insieme (Set)



- Un insieme (set) è:
 - una collezione non ordinata di elementi senza duplicati
 - Esempio:
 - Le vocali: {'a', 'e', 'i', 'o', 'u'}
-
-
-

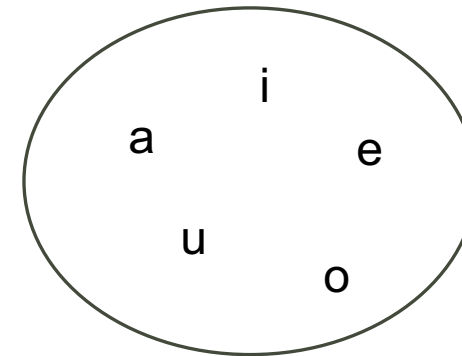


È un insieme

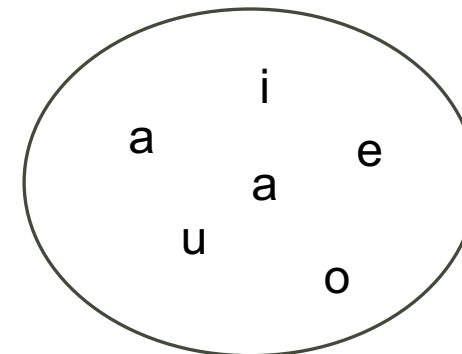
Insieme (Set)



- Un insieme (set) è:
 - una collezione non ordinata di elementi senza duplicati
 - Esempio:
 - Le vocali: {'a', 'e', 'i', 'o', 'u'}
- Controesempio:
 - {'a', 'e', 'i', 'o', 'u', 'a'}
 -



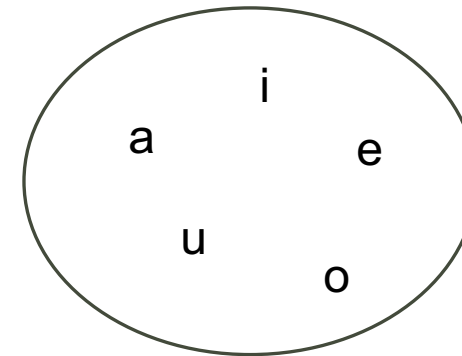
È un insieme



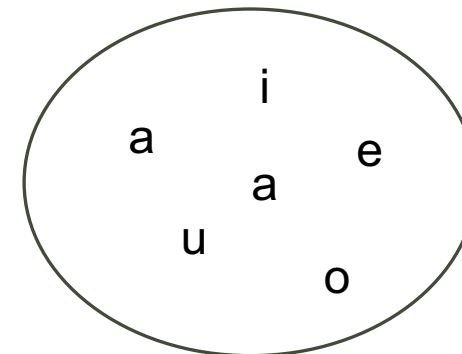
Insieme (Set)



- Un insieme (set) è:
 - una collezione non ordinata di elementi senza duplicati
 - Esempio:
 - Le vocali: {'a', 'e', 'i', 'o', 'u'}
- Controesempio:
 - {'a', 'e', 'i', 'o', 'u', 'a'}
 - Contiene 'a' due volte



È un insieme



Non è un insieme

Insieme (Set) in C I

1.

•

2.

•

•

•

•

•

•

•

- toglie un elemento all'insieme (restituisce 0 se l'elemento non era presente, 1 altrimenti)

Insieme (Set) in C I

1. Nome del tipo

- `Set (di Element)`

2. Signature e semantica (astratta) delle operazioni fondamentali:

- `Set mkSet()`
 - Restituisce un insieme vuoto
- `dsSet (Set*)`
 - Distrugge l'insieme, recuperando la memoria
- `_Bool setAdd(Set, const Element)`
 - Aggiunge un elemento all'insieme (restituisce 0 se l'elemento era già presente, 1 altrimenti)
- `_Bool setRemove(Set, const Element)`
 - toglie un elemento all'insieme (restituisce 0 se l'elemento non era presente, 1 altrimenti)

Insieme (Set) in C II



2. (continuazione) signature e semantica (astratta) delle operazioni fondamentali:

- `_Bool setMember(const Set, const Element)`
 - Controlla se un elemento appartiene all'insieme
- `Bool isEmptySet(const Set)`
 - Controlla se l'insieme è vuoto
- `int setSize(const Set)`
 - Restituisce il numero di elementi presenti nell'insieme
- `Element setExtract(Set)`
 - Toglie e restituisce un elemento a caso dall'insieme
- `_Bool setEquals(const Set, const Set)`
 - Controlla se due insiemi sono uguali

Insieme (Set) in C III



2. (continuazione) signature e semantica (astratta) delle operazioni fondamentali:

- `_Bool subsetEq(const Set, const Set)`
 - Controlla se il primo insieme è incluso nel secondo
- `_Bool subset(const Set, const Set)`
 - Controlla se il primo insieme è strettamente incluso nel secondo
- `Set setUnion(const Set, const Set)`
 - Restituisce l'unione di due insiemi
- `Set setIntersection(const Set, const Set)`
 - Restituisce l'intersezione di due insiemi
- `Set setSubtraction(const Set, const Set)`
 - Restituisce l'insieme ottenuto dal primo togliendo gli elementi del secondo

Queue (Coda) in C IV



3. Implementazioni:

- Con liste:
 - Insieme di dimensione illimitato
 - Mi ricordo di quanti elementi ci sono
 - Simile alla pila
- Con array (statico):
 - Insieme di dimensione limitato
 - Mi ricordo di quanti elementi ci sono
 - Simile alla pila

Queue (Coda) in C IV



3. Implementazioni:

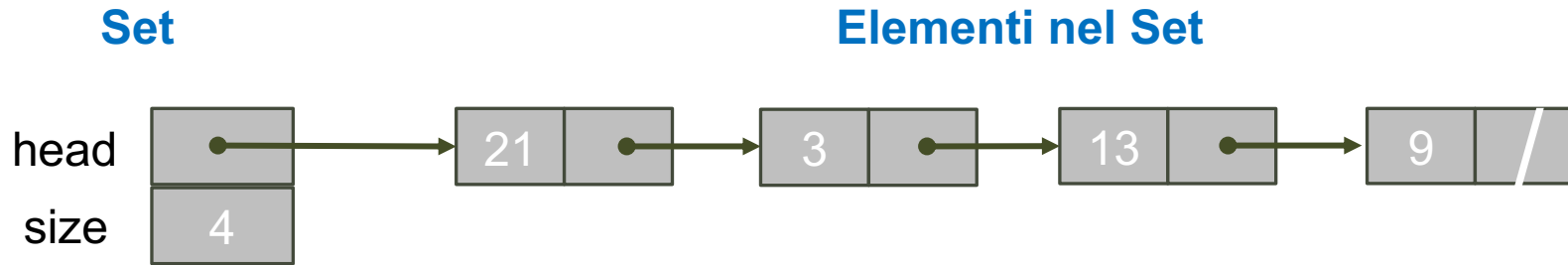
- Con liste:
 - Insieme di dimensione illimitato
 - Mi ricordo di quanti elementi ci sono
 - Simile alla pila
- Con array (statico):
 - Insieme di dimensione limitato
 - Mi ricordo di quanti elementi ci sono
 - Simile alla pila

Vediamo solo
questa

Esempio: Insieme di `int`

- Nome del tipo:
 - `typedef struct intSet *IntSetADT`
 - Definito in `intSetADT.h` insieme alle signature delle funzioni
- Come procediamo?
 - In modo iterativo e incrementale applicando la metodologia TDD
 - Sviluppiamo un'implementazione per la specifica `intSetADT.h` partendo da una test suite `test_intSetADT.c`
 - Consideriamo solo implementazione con liste:
 - Modulo `intLinkedListSetADT.c`
 - Abbozziamo solo alcune funzioni principali

Insieme di `int` con Liste



- Implementazione con liste linkate:
 - Teniamo traccia del numero di elementi
 - Semplifica alcune operazioni

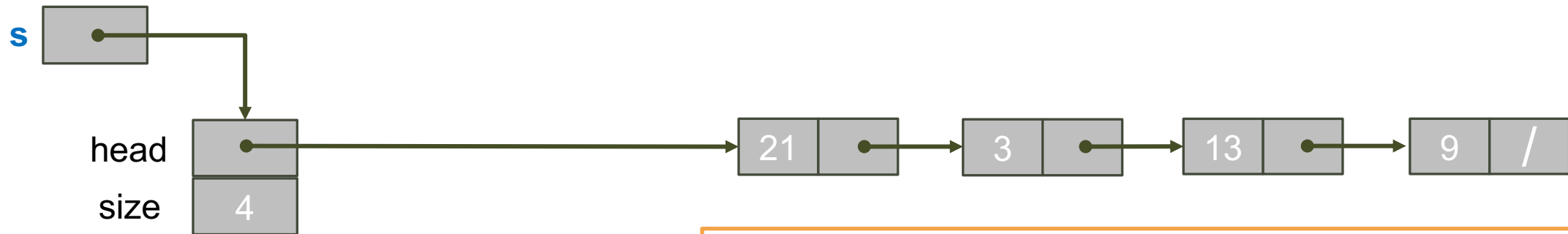
Elementi nel Set, i.e., Lista

```
typedef struct listNode ListNode,  
        *ListNodePtr;  
  
struct listNode {  
    int data;  
    ListNodePtr next;  
};
```

Set

```
struct intSet {  
    ListNodePtr head;  
    int size;  
};
```


Insieme di set con Liste: setAdd()

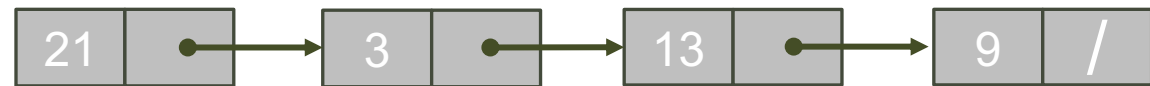
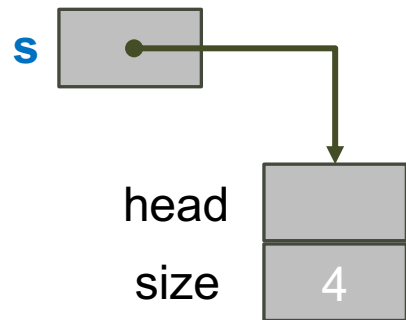


- Aggiunge un elemento all'insieme
 - Restituisce:
 - 0 se l'elemento era già presente
 - 1 altrimenti
 - Basta un'inserzione in testa?
 -

```
_Bool setAdd(Set s, const int e) {
```

```
}
```

Insieme di set con Liste: setAdd()

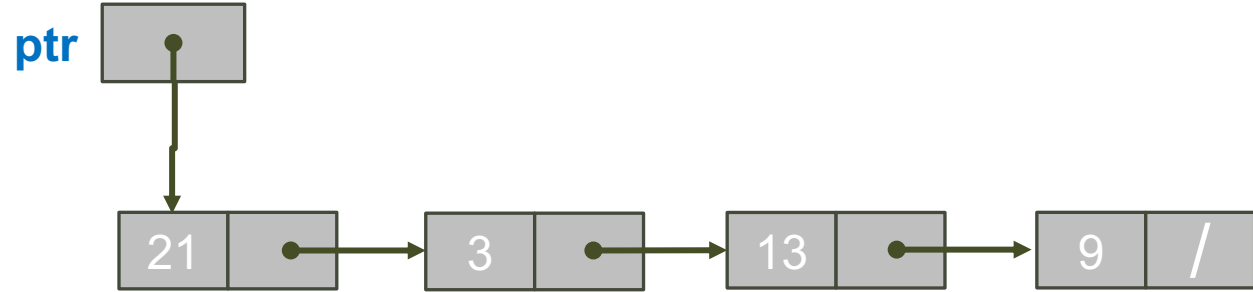
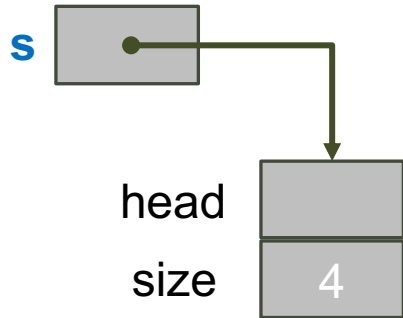


- Aggiunge un elemento all'insieme
 - Restituisce:
 - 0 se l'elemento era già presente
 - 1 altrimenti
 - Basta un'inserzione in testa?
 - No: controllo duplicati

```
_Bool setAdd(Set s, const int e) {
```

```
}
```

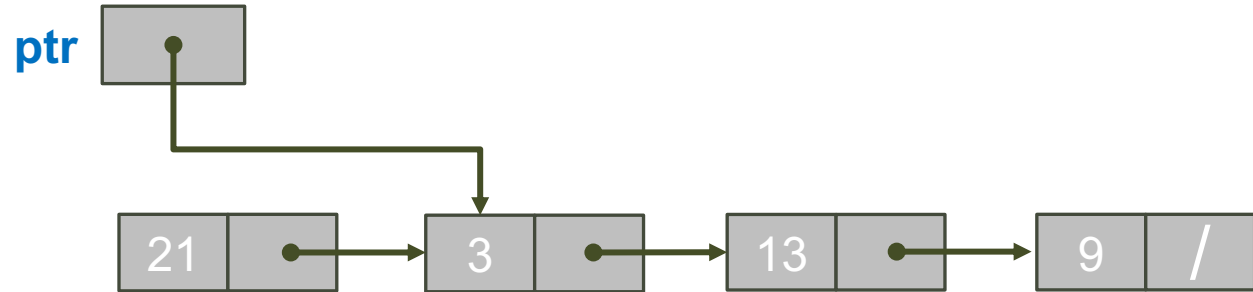
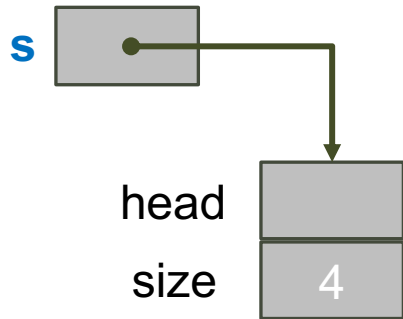
Insieme di set con Liste: setAdd()



- Aggiunge un elemento all'insieme
 - Restituisce:
 - 0 se l'elemento era già presente
 - 1 altrimenti
 - Basta un'inserzione in testa?
 - No: controllo duplicati

```
_Bool setAdd(Set s, const int e) {
    // Visita la lista cercando e
    ListNodePtr ptr = s->head;
    while(ptr != NULL) {
        if(ptr->data == e) return 0; // Trovato, rest. 0
        ptr = ptr->next;
    }
}
```

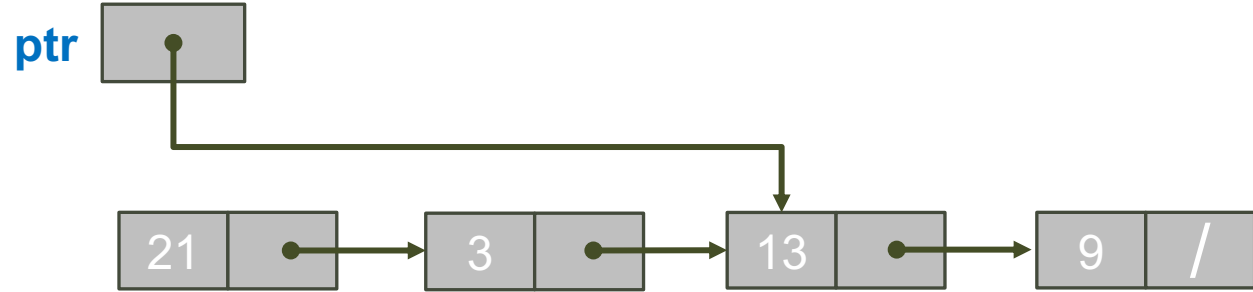
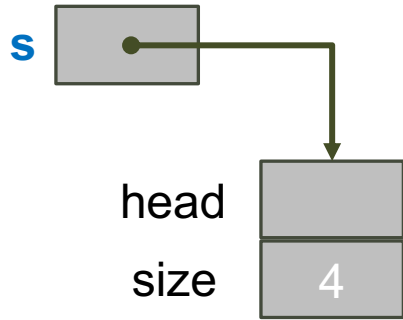
Insieme di set con Liste: setAdd()



- Aggiunge un elemento all'insieme
 - Restituisce:
 - 0 se l'elemento era già presente
 - 1 altrimenti
 - Basta un'inserzione in testa?
 - No: controllo duplicati

```
_Bool setAdd(Set s, const int e) {
    // Visita la lista cercando e
    ListNodePtr ptr = s->head;
    while(ptr != NULL) {
        if(ptr->data == e) return 0; // Trovato, rest. 0
        ptr = ptr->next;
    }
}
```

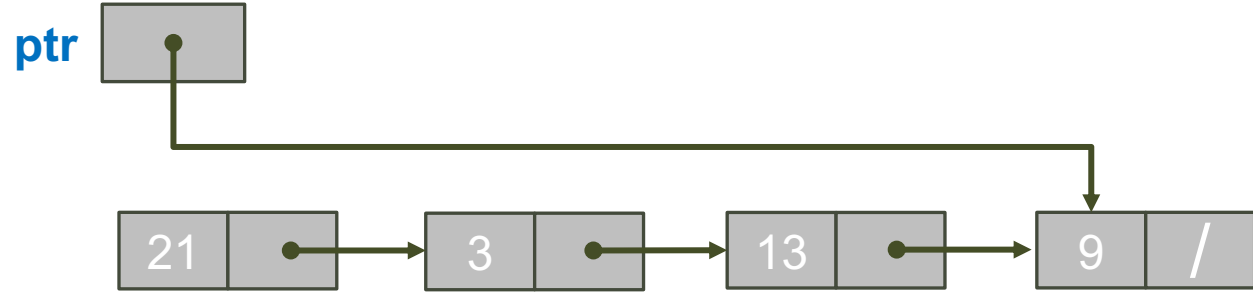
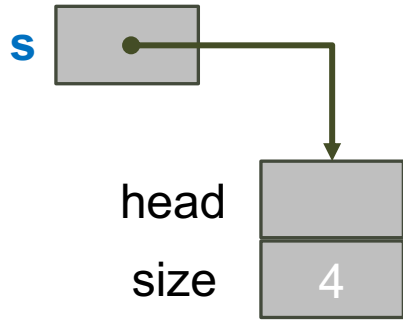
Insieme di set con Liste: setAdd()



- Aggiunge un elemento all'insieme
 - Restituisce:
 - 0 se l'elemento era già presente
 - 1 altrimenti
 - Basta un'inserzione in testa?
 - No: controllo duplicati

```
_Bool setAdd(Set s, const int e) {  
    // Visita la lista cercando e  
    ListNodePtr ptr = s->head;  
    while(ptr != NULL) {  
        if(ptr->data == e) return 0; // Trovato, rest. 0  
        ptr = ptr->next;  
    }  
}
```

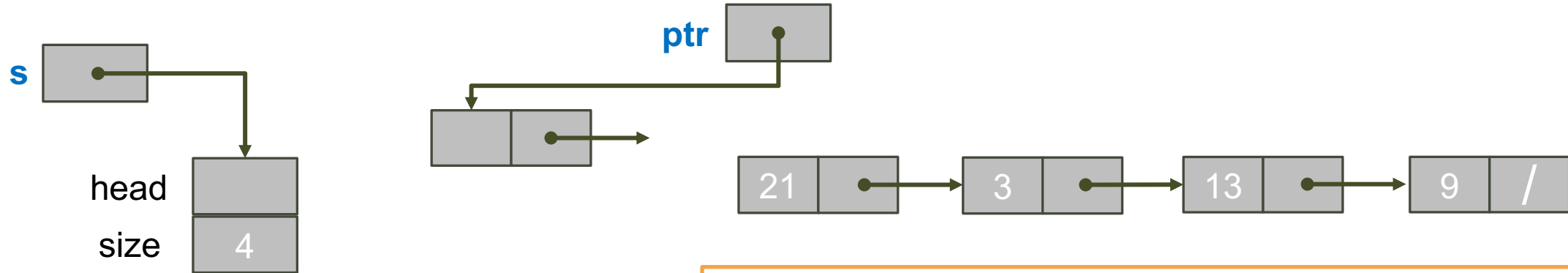
Insieme di set con Liste: setAdd()



- Aggiunge un elemento all'insieme
 - Restituisce:
 - 0 se l'elemento era già presente
 - 1 altrimenti
 - Basta un'inserzione in testa?
 - No: controllo duplicati

```
_Bool setAdd(Set s, const int e) {  
    // Visita la lista cercando e  
    ListNodePtr ptr = s->head;  
    while(ptr != NULL) {  
        if(ptr->data == e) return 0; // Trovato, rest. 0  
        ptr = ptr->next;  
    }  
}
```

Insieme di set con Liste: setAdd()

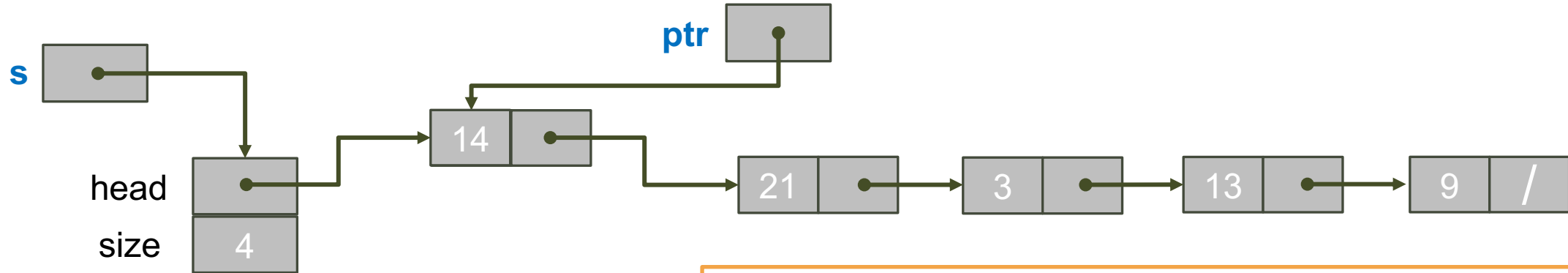


- Aggiunge un elemento all'insieme
 - Restituisce:
 - 0 se l'elemento era già presente
 - 1 altrimenti
 - Basta un'inserzione in testa?
 - No: controllo duplicati

```
_Bool setAdd(Set s, const int e) {
    // Visita la lista cercando e
    ListNodePtr ptr = s->head;
    while(ptr != NULL) {
        if(ptr->data == e) return 0; // Trovato, rest. 0
        ptr = ptr->next;
    }
    // Non trovato
    ptr = malloc(sizeof(*ptr));
    if (ptr == NULL) return 0; // malloc() fallita?

}
```

Insieme di set con Liste: setAdd()

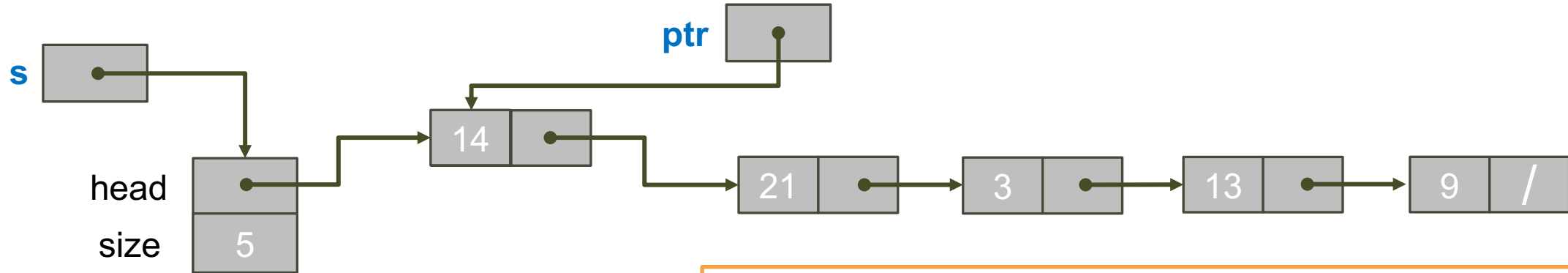


- Aggiunge un elemento all'insieme
 - Restituisce:
 - 0 se l'elemento era già presente
 - 1 altrimenti
 - Basta un'inserzione in testa?
 - No: controllo duplicati

```

_Bool setAdd(Set s, const int e) {
    // Visita la lista cercando e
    ListNodePtr ptr = s->head;
    while(ptr != NULL) {
        if(ptr->data == e) return 0; // Trovato, rest. 0
        ptr = ptr->next;
    }
    // Non trovato
    ptr = malloc(sizeof(*ptr));
    if (ptr == NULL) return 0; // malloc() fallita?
    ptr->next = s->head; s->head = ptr; // Ins. in testa
    ptr->data = e;
}
  
```


Insieme di set con Liste: setAdd()



- Aggiunge un elemento all'insieme
 - Restituisce:
 - 0 se l'elemento era già presente
 - 1 altrimenti
 - Basta un'inserzione in testa?
 - No: controllo duplicati

```

_Bool setAdd(Set s, const int e) {
    // Visita la lista cercando e
    ListNodePtr ptr = s->head;
    while(ptr != NULL) {
        if(ptr->data == e) return 0; // Trovato, rest. 0
        ptr = ptr->next;
    }
    // Non trovato
    ptr = malloc(sizeof(*ptr));
    if (ptr == NULL) return 0; // malloc() fallita?
    ptr->next = s->head; s->head = ptr; // Ins. in testa
    ptr->data = e;
    s->size++;
    return s;
}
  
```

Insieme Ordinato di `int`



- Ogni aggiunta:
 - Richiede la vista completa della lista per il controllo dei duplicati
 - Complessità computazionale: $O(N)$ per insieme con N elementi
- Possiamo migliorare il caso medio?
 - Ordinando l'insieme
 - Al primo elemento più grande (se ordine crescente) possiamo interrompere ricerca
 - Nota caso peggiore (elemento non presente) complessità rimane uguale

Insieme Ordinato (Sorted Set) in C I

1. Nome del tipo

- SortedSet (di Element)

2.

- -
 -
 -
 -
 -
 -
 -
 -

Insieme Ordinato (Sorted Set) in C I

1. Nome del tipo

- `SortedSet` (di `Element`)

2. Signature e semantica (astratta) delle operazioni: tutte quelle di `Set`

- -
 -
 -
 -
 -
 -
 -

Insieme Ordinato (Sorted Set) in C I

1. Nome del tipo

- `SortedSet` (di `Element`)

2. Signature e semantica (astratta) delle operazioni: tutte quelle di `Set`

- Più le seguenti 4 operazioni
 - `Element ssetMin(Set)`
 - Restituisce l'elemento minimo
 - `Element ssetMax(Set)`
 - Restituisce l'elemento massimo
 - `Element ssetExtractMin(Set)`
 - Toglie e restituisce l'elemento minimo
 - `Element ssetExtractMax(Set)`
 - Toglie e restituisce l'elemento massimo

Esempio: Insieme Ordinato di `int`

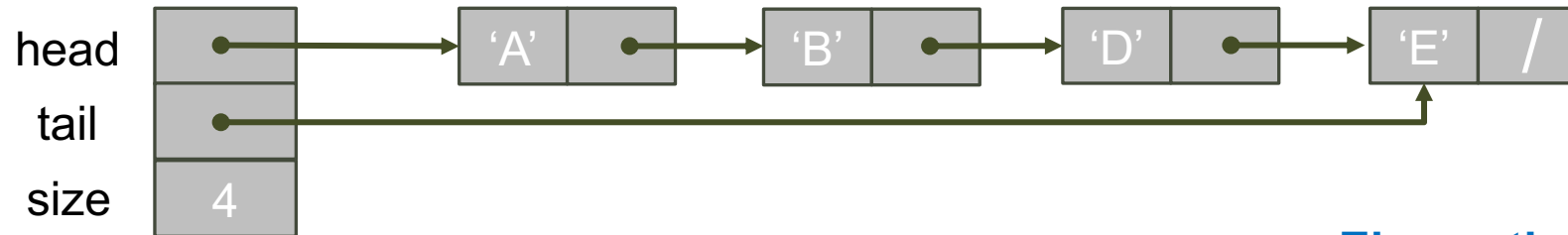
- Nome del tipo:
 - `typedef struct intSortedSet *IntSortedSetADT`
 - Definito in `intSortedSetADT.h` insieme alle signature delle funzioni
- Come procediamo?
 - In modo iterativo e incrementale applicando la metodologia TDD
 - Sviluppiamo un'implementazione per la specifica `intSortedSetADT.h` partendo da una test suite `test_intSortedSetADT.c`
 - Consideriamo solo implementazione con liste:
 - Modulo `intLinkedListSortedSetADT.c`
 - Abbozziamo solo alcune funzioni principali

Insieme Ordinato di `int` con Liste



Sorted Set

Elementi nel Set Sorted



- Implementazione con liste linkate:
 - Teniamo traccia
 - Del numero di elementi
 - Della coda (double ended list)
 - Semplifica alcune operazioni

Elementi nel Sorted Set, i.e., Lista

```
typedef struct listNode ListNode,
    *ListNodePtr;

struct listNode {
    int data;
    ListNodePtr next;
};
```

Sorted Set

```
struct intSet {
    ListNodePtr head, tail;
    int size;
};
```